

PHƯƠNG PHÁP KHỬ GAUSS

1.1. Lập trình thuật toán.

1.1.1. Mô tả.

Cú pháp gọi hàm:

```
% x = gaussian_elimination(A,b)
```

Tham số thuật toán:

```
% -----  
% INPUT:  
% -----  
% A      - (n×n double) matrix  
% b      - (n×1 double) vector  
% -----  
% OUTPUT:  
% -----  
% x      - (n×1 double) nghiệm của Ax=b
```

1.1.2. Thuật toán.

```
function x = gaussian_elimination(A,b)  
    n = size(A,1);           % xác định n là kích thước ma trận  
    A = [A,b];               % tạo ma trận mở rộng  
    singular = false;        % biến theo dõi ma trận đơn  
  
    % quá trình khử  
    for i = 1:(n-1)  
        % xác định hàng cố định  
        p = i:1:n;  
        p(A(p,i) == 0) = [];  
        p = min(p);  
  
        % nếu tất cả các phần tử trực xoay trong cột đều bằng 0  
        % thì ma trận là ma trận đơn, thoát vòng lặp  
        if max(abs(A(:,i))) <= eps  
            singular = true;  
            break  
        end  
        % nếu p không bằng i, đổi vị trí hàng thứ i và hàng thứ p  
        if p ~= i  
            Ai = A(i,:);  
            Ap = A(p,:);  
            A(i,:) = Ap;  
            A(p,:) = Ai;  
        end  
        % elementary row operation  
        for j = (i+1):n  
            A(j,:) = A(j, :)-(A(j,i)/A(i,i))*A(i,:);  
        end  
    end  
  
    % nếu phần tử hàng dưới cùng bên phải = 0 hay cả hàng = 0  
    % thì ma trận là ma trận đơn, thoát vòng lặp  
    if abs(A(n,n)) <= eps  
        singular = true;  
    end  
  
    % tạo vector lưu lời giải  
    x = zeros(n,1);
```

```

% thế ngược để giải Ax = b nếu ma trận không phải ma trận đơn
if ~singular
    x(n) = A(n,n+1)/A(n,n);
    for i = (n-1):-1:1
        S = 0;
        for j = (i+1):n
            S = S+A(i,j)*x(j);
        end
        x(i) = (A(i,n+1)-S)/A(i,i);
    end
end

% in cảnh báo nếu ma trận đơn
if singular
    warning("Matrix is singular to working precision.")
end
end

```

1.3. Bài tập.

7. Use Algorithm 6.1 and Maple with *Digits*:= 10 to solve the following linear systems.

- | | |
|--|---|
| <p>a. $\frac{1}{4}x_1 + \frac{1}{5}x_2 + \frac{1}{6}x_3 = 9,$
 $\frac{1}{3}x_1 + \frac{1}{4}x_2 + \frac{1}{5}x_3 = 8,$
 $\frac{1}{2}x_1 + x_2 + 2x_3 = 8.$</p> | <p>b. $3.333x_1 + 15920x_2 - 10.333x_3 = 15913,$
 $2.222x_1 + 16.71x_2 + 9.612x_3 = 28.544,$
 $1.5611x_1 + 5.1791x_2 + 1.6852x_3 = 8.4254.$</p> |
| <p>c. $x_1 + \frac{1}{2}x_2 + \frac{1}{3}x_3 + \frac{1}{4}x_4 = \frac{1}{6},$
 $\frac{1}{2}x_1 + \frac{1}{3}x_2 + \frac{1}{4}x_3 + \frac{1}{5}x_4 = \frac{1}{7},$
 $\frac{1}{3}x_1 + \frac{1}{4}x_2 + \frac{1}{5}x_3 + \frac{1}{6}x_4 = \frac{1}{8},$
 $\frac{1}{4}x_1 + \frac{1}{5}x_2 + \frac{1}{6}x_3 + \frac{1}{7}x_4 = \frac{1}{9}.$</p> | <p>d. $2x_1 + x_2 - x_3 + x_4 - 3x_5 = 7,$
 $x_1 + 2x_3 - x_4 + x_5 = 2,$
 $-2x_2 - x_3 + x_4 - x_5 = -5,$
 $3x_1 + x_2 - 4x_3 + 5x_5 = 6,$
 $x_1 - x_2 - x_3 - x_4 + x_5 = 3.$</p> |

Bài 7. Áp dụng phương pháp khử Gauss để giải hệ phương trình.

a.

```
>> A = [1/4 1/5 1/6; 1/3 1/4 1/5; 1/2 1 2]
```

A =

```

0.2500    0.2000    0.1667
0.3333    0.2500    0.2000
0.5000    1.0000    2.0000

```

```
>> b = [9; 8; 8]
```

b =

```

9
8
8

```

```
>> x = gaussian_elimination(A,b)
```

x =

```

-227.0769
476.9231
-177.6923

```

b. >> A = [3.333 15920 -10.333; 2.222 16.71 9.612; 1.5611 5.1791 1.6852]

A =

1.0e+04 *	0.000333300000000	1.59200000000000	-0.001033300000000
	0.000222200000000	0.00167100000000	0.000961200000000
	0.000156110000000	0.00051791000000	0.000168520000000

>> b = [15913; 28.544; 8.4254]

b =

1.0e+04 *	1.59130000000000
	0.00285440000000
	0.00084254000000

>> x = gaussian_elimination(A,b)

x =

0.999999999999616
1.00000000000000
1.000000000000148

c.

>> A = [1 1/2 1/3 1/4; 1/2 1/3 1/4 1/5; 1/3 1/4 1/5 1/6; 1/4 1/5 1/6 1/7]

A =

1.00000000000000	0.50000000000000	0.33333333333333	0.25000000000000
0.50000000000000	0.33333333333333	0.25000000000000	0.20000000000000
0.33333333333333	0.25000000000000	0.20000000000000	0.16666666666667
0.25000000000000	0.20000000000000	0.16666666666667	0.142857142857143

>> b = [1/6; 1/7; 1/8; 1/9]

b =

0.166666666666667
0.142857142857143
0.125000000000000
0.111111111111111

>> x = gaussian_elimination(A,b)

x =

-0.031746031746026
0.595238095238029
-2.380952380952220
2.777777777777673

d.

>> A = [2 1 -1 1 -3; 1 0 2 -1 1; 0 -2 -1 1 -1; 3 1 -4 0 5; 1 -1 -1 -1 1]

A =

2	1	-1	1	-3
1	0	2	-1	1
0	-2	-1	1	-1
3	1	-4	0	5
1	-1	-1	-1	1

```
>> b = [7; 2; -5; 6; 3]
b =
    7
    2
   -5
    6
    3
>> x = gaussian_elimination(A,b)
x =
    1.918128654970760
    1.964912280701754
   -0.988304093567251
   -3.192982456140350
   -1.134502923976608
```

17. Use the hybrid method described in Exercise 16 and two-digit rounding arithmetic to solve the systems in Exercise 3.

$$x_1 = 1, x_2 = -1, x_3 = 3.)$$

$$\begin{aligned} \text{a. } 4x_1 - x_2 + x_3 &= 8, \\ 2x_1 + 5x_2 + 2x_3 &= 3, \\ x_1 + 2x_2 + 4x_3 &= 11. \end{aligned}$$

$$\begin{aligned} \text{b. } 4x_1 + x_2 + 2x_3 &= 9, \\ 2x_1 + 4x_2 - x_3 &= -5, \\ x_1 + x_2 - 3x_3 &= -9. \end{aligned}$$

Bài 17.

a.

```
>> A = [4 -1 1; 2 5 2; 1 2 4]
```

A =

```
    4    -1     1
    2     5     2
    1     2     4
```

```
>> b = [8; 3; 11]
```

b =

```
    8
    3
   11
```

```
>> x = gaussian_elimination(A,b)
```

x =

```
    1
   -1
    3
```

b.

```
>> A = [4 1 2; 2 4 -1; 1 1 -3]
```

A =

```
    4     1     2
    2     4    -1
    1     1    -3
```

```
>> b = [9; -5; -9]
```

b =

```
    9
   -5
   -9
```

```
>> x = gaussian_elimination(A,b)
```

x =

```
    1
   -1
    3
```

PHƯƠNG PHÁP PHÂN TÍCH LU

2.1. Lập trình thuật toán.

2.1.1. Giải hệ phương trình.

```
A = input('Nhập các phần tử ma trận A: \n');
% A= [7 -3 1; 2 9 -3; 5 4 11] % Nhập ma trận hệ số A. HPT: AX=B.
B = input('Nhập vector B: \n');
%B = [21; 37; 15] % Nhập ma trận hằng số B

[L,U] = doolittle(A); % Đổi lu_pivot(A) thành crout(A) |
% Phân tích A thành tích của L*U % với trường hợp ma trận đối xứng đặc biệt
disp('Ma trận tam giác dưới L:');
disp(L);
disp('Ma trận tam giác trên U:');
disp(U);

P = [L B]; % Lập hpt P: LY = B
[ row,col] = size(P);
for m=1:row
    a = P(m,m); % Báo lỗi nếu không đúng dạng ma trận
    if a==0
        disp('Không thể áp dụng phương pháp phân tích LU');
        return
    end
    P(m,:) = P(m,+)/a;
end
s=0;
for m=1:row-1 % Giải hpt tìm Y
    for k=row-m:row-1
        s = s + P(m+1,col-k-1) * P(col-k-1,col);
        P(m+1,col) = P(m+1,col)- s;
        s = 0;
    end
end
Y = P(:,col);
disp('Giải LY = B được Y là ');
disp(Y);
Q = [U Y]; % Lập hpt Q: UX = Y
for m=1:row
    b = Q(m,m); % Báo lỗi nếu không đúng dạng ma trận
    if b==0
        disp('Không thể áp dụng phương pháp phân tích LU');
        return
    end
    Q(m,:) = Q(m,+)/b;
end
s = 0;
for m=row:-1:2 % Giải hpt tìm X
    for k=m+1:col
        s = s + Q(m-1,k-1) * Q(k-1,col);
        Q(m-1,col) = Q(m-1,col)- s;
        s = 0;
    end
end
X= Q(:,col);
disp('Nghịệm của hệ phương trình là X:'); disp(X);
```

2.1.2. Phương pháp phân tích Crout.

```
function [L, U] = crout(A)
[m, n] = size(A);
L = zeros(n, n);
U = eye(n);
assert(m == n)
assert(length(eigs(A)) == n)
for j = 1:n
    L(j:n, j) = A(j:n, j) - L(j:n, 1:j-1) * U(1:j-1, j);
    U(j, j+1:n) = 1 / L(j, j) * (A(j, j+1:n) - L(j, 1:j-1) * U(1:j-1, j+1:n));
end
end
```

2.1.3. Phương pháp phân tích Doolittle.

```
function [L, U] = doolittle(A)
[m, n] = size(A);
L = eye(n);
U = zeros(m, n);
assert(m == n)
assert(length(eigs(A)) == n)
for i = 1:n
    U(i, i:n) = A(i, i:n) - L(i, 1:i-1) * U(1:i-1, i:n);
    L(i+1:n, i) = 1/U(i, i) * (A(i+1:n, i) - L(i+1:n, 1:i-1) * U(1:i-1, i));
end
end
```

2.1.4. Phương pháp Cholesky

```
function L = cholesky(A)
[~, n] = size(A);
% Kiểm tra tính đối xứng của ma trận
t = issymmetric(A);
if t == 0
    fprintf('\nMa tran khong doi xung.\nKhong the thuc hien thuat toan.');
```

```
    return
end
% kiểm tra ma trận đơn \ không đủ điều kiện.
r = rank(A);
if r < n
    fprintf('\nHạng của ma trận. Không thể phân tích Cholesky đầy đủ.\n');
```

```
    return
end
% lặp
L = zeros(n,n);
for j = 1:n
    sum = 0;
    for k = 1:j-1
        sum = sum + (L(j,k))^2;
    end
    L(j,j) = sqrt(A(j,j)-sum);
    for i = j+1:n
        sum = 0;
        for k = 1:j-1
            sum = sum + L(i,k)*L(j,k);
        end
        L(i,j) = (A(i,j)-sum)/L(j,j);
    end
end
end
```

Phương pháp Cholesky chỉ áp dụng cho ma trận đối xứng có hạng ma trận thỏa mãn. Phương pháp này tìm được ma trận tam giác dưới L, ma trận tam giác trên là ma trận chuyển vị của ma trận tam giác dưới. Khi áp dụng phương pháp Cholesky sẽ cần thay đổi chương trình bên dưới để phù hợp với ma trận L và ma trận U mới.

2.3. Bài tập.

VD. Trường hợp ma trận không đối xứng, phân tích bằng phương pháp Doolittle.

```
>> alu
```

Nhap he so ma tran A:

```
[7 -3 1; 2 9 -3; 5 4 11]
```

Nhap vector B:

```
[21; 37; 15]
```

Ma tran tam giac duoi L la

1.0000	0	0
0.2857	1.0000	0
0.7143	0.6232	1.0000

Ma tran tam giac tren U la

7.0000	-3.0000	1.0000
0	9.8571	-3.2857
0	0	12.3333

Nghiem Y tim duoc la

```
21.0000  
31.0000  
-19.3188
```

Nghiem X cua he phuong trinh la

```
4.3478  
2.6228  
-1.5664
```

$$7. \quad 9x_1 + 6x_2 + 12x_3 = 17.4$$

$$6x_1 + 13x_2 + 11x_3 = 23.6$$

$$12x_1 + 11x_2 + 26x_3 = 30.8$$

Bài 7. Giải hệ phương trình bằng phương pháp phân tích Crout với ma trận đối xứng.

>> alu

Nhap he so ma tran A:

[9 6 12; 6 13 11; 12 11 26]

Nhap vector B:

[17.4; 23.6; 30.8]

Kết quả:

Ma tran tam giac duoi L la

9	0	0
6	9	0
12	3	9

Ma tran tam giac tren U la

1.0000	0.6667	1.3333
0	1.0000	0.3333
0	0	1.0000

Nghiem Y tim duoc la

1.9333
1.3333
0.4000

Nghiem X cua he phuong trinh la

0.6000
1.2000
0.4000

PHƯƠNG PHÁP LẬP GAUSS - SEIDEL

3.1. Lập trình thuật toán.

```
clc;
close all;
A= [ 2 3 6; 1 7 3; 5 1 3]           % Nhập ma trận A
B = [3; 9; 7]                       % Nhập ma trận B
P= [A B];                           % tạo ma trận mở rộng P
[row col] = size(P);                 % tính kích thước ma trận P

for i = 1:row-1                     % Tìm xem ma trận tam giác dưới bằng 0
    if P(i,i) == 0                   % nếu các phần tử thuộc đường chéo bằng 0
        disp('Không thể khử Gauss. Xếp lại thứ tự các hàng !!!')
        return
    end
    a=P(i,i);
    P(i,:)= P(i,+)/a;
    for j=i+1:row
        P(j,:)= P(j,)- P(j,i)* P(i,);
    end
end
a=P(row,row);
P(row,:)= P(row,+)/a;
for i=row:-1:2                     % Tìm ma trận tam giác trên bằng 0
    for j=i-1:-1:1
        P(j,:)= P(j,)- P(j,i)* P(i,);
    end
end
disp('Lời giải tìm được là:')
P(:,col)
```

3.2. Bài tập.

VD:

A =

2	3	6
1	7	3
5	1	3

B =

3
9
7

The required solution is:

ans =

1.545454545454545
1.363636363636364
-0.696969696969697

Bài 7.

A =

5	-2	9
-2	10	-2
0	-2	15

B =

18
-60
128

The required solution is:

ans =

-12.889518413597735
-7.059490084985836
7.592067988668555

PHƯƠNG PHÁP LẬP GAUSS - SEIDEL

4.1. Lập trình thuật toán.

```
function X = gauss_seidel( A ,B )
A = input ( 'Enter matrix A : \n')           % Nhập ma trận A
% kiểm tra ma trận A vuông hay không
[na , ma] = size (A);
if na ~= ma
    disp('ERROR: Matrix A phải là ma trận vuông')
    return
end
B = input ( 'Enter matrix B : ')             % Nhập ma trận B
% kiểm tra ma trận B có phải 1 cột hay không
[nb , mb ] = size (B);
if nb ~= na || mb~=1
    disp( 'ERROR: Matrix B phải là ma trận cột')
    return
end
% Tách ma trận A thành ma trận tam giác trên và ma trận tam giác dưới
% A = D + L + U
D = diag(diag(A));
L = tril(A)- D;
U = triu(A)- D
% kiểm tra điều kiện hội tụ của Gauss-Seidel method
e = max(eig(-inv(D+L)*(U)));
if abs(e) >= 1
    disp ( 'Trị riêng lớn nhất của ma trận lập lớn hơn 1')
    disp ( 'Không thể hội tụ.')
    return
end
% tạo giá trị đoán ban đầu cho X
% mặc định X0 = [ 1 1 .... 1]
r = input ( 'Nhập giá trị bắt đầu X0? (y/n): ', 's');
switch r
    case 'y'
        % nhập giá trị bắt đầu
        X0 = input('Enter initial guess for X :\n')
        % kiểm tra giá trị bắt đầu
        [nx, mx] = size(X0);
        if nx ~= na || mx ~= 1
            disp( 'ERROR: Check input')
            return
        end
    otherwise
        X0 = ones(na,1);
end
% sai số cho phép của kết quả cuối cùng
t = input ( 'Nhập sai số cho phép cho kết quả nghiệm cuối cùng: ');
tol = t*ones(na,1);
k= 1;
X( : , 1 ) = X0;
err= 1000000000*rand(na,1); % khởi tạo sai số đầu tiên để thực hiện lặp
while sum(abs(err) >= tol) ~= zeros(na,1)
    X ( : ,k+ 1 ) = -inv(D+L)*(U)*X( : ,k) + inv(D+L)*B; % CT Gauss-Seidel
    err = X( :,k+1) - X( :, k); % tính sai số
    k = k + 1;
end
fprintf ( 'The final answer obtained after %g iterations is \n', k)
X( : ,k)
```

4.2. Bài tập.

$$\begin{aligned} 7. \quad 5x_1 - 2x_2 &= 18 \\ -2x_1 + 10x_2 - 2x_3 &= -60 \\ -2x_2 + 15x_3 &= 128 \end{aligned}$$

Bài 7:

Enter matrix A :

[5 -2 0; -2 10 -2; 0 -2
15]

A =

5	-2	0
-2	10	-2
0	-2	15

Enter matrix B :

[18; -60; 128]

B =

18
-60
128

U =

0	-2	0
0	0	-2
0	0	0

Any initial guess for X? (y/n): n

Enter the error allowed in final answer: 1e-4

The final answer obtained after 8 iterations is

ans =

1.999994476636708

-4.000001472896877

7.999999803613750

ans (i) =

Columns 1 through 4

1.000000000000000	4.000000000000000	1.600000000000000	1.957333333333334
1.000000000000000	-4.999999999999999	-4.106666666666666	-4.011377777777777
1.000000000000000	7.866666666666667	7.985777777777778	7.998482962962963

Columns 5 through 8

1.995448888888889	1.999514548148148	1.999948218469136	1.999994476636708
-4.001213629629629	-4.000129453827160	-4.000013808408230	-4.000001472896877
7.999838182716050	7.999982739489712	7.999998158878903	7.999999803613750

PHƯƠNG PHÁP LẬP JACOBI

5.1. Lập trình thuật toán.

```
function [x, it] = jacobi_method(A, b)
%%%%%% jacobi method %%%%%%%

% kiểm tra kích thước của vector và ma trận
s=size(A); % lấy kích thước của A
rows=s(1); % số hàng của ma trận A
cols=s(2); % số cột của ma trận A

S=zeros(size(A));

for i=1:cols;
    S(i,i)=A(i,i);
end

S
T=S-A
x=zeros(size(b))
xnew=zeros(size(b));

for it=1:10
    for i=1:cols
        xnew(i)=b(i);
        for j=1:rows
            xnew(i)=xnew(i)+T(i,j)*x(j);
        end
        xnew(i)=xnew(i)/S(i,i);
    end
    x=xnew;
    disp('iteration ')
    it
    x
    pause
end;
```

5.2. Bài tập.

```
>> A = [5 -2 3; -3 9 1; 2 -1 -7]      >> b = [-1; 2; 3]
A =                                     b =
     5     -2      3                  -1
    -3      9      1                   2
     2     -1     -7                   3

>> [xn, count] = jacobi_method(A, b)

S =                                     T =                                     x =
     5      0      0                   0      2     -3                   0
     0      9      0                   3      0     -1                   0
     0      0     -7                  -2      1      0                   0
```

iteration	iteration	iteration	iteration	iteration
it =	it =	it =	it =	it =
1	3	5	7	9
x =	x =	x =	x =	x =
-0.2000	0.1917	0.1854	0.1861	0.1861
0.2222	0.3284	0.3293	0.3313	0.3312
-0.4286	-0.4159	-0.4244	-0.4226	-0.4227

iteration	iteration	iteration	iteration	iteration
it =	it =	it =	it =	it =
2	4	6	8	10
x =	x =	x =	x =	x =
0.1460	0.1809	0.1863	0.1861	0.1861
0.2032	0.3323	0.3312	0.3312	0.3312
-0.5175	-0.4207	-0.4226	-0.4227	-0.4227

So sánh kết quả ta thấy kết quả tính tay và tính máy trùng khớp nhau, vậy thuật toán đã lập trình đúng.

$$\begin{aligned}
 x_1 &= \frac{-1}{5} + \frac{2}{5}x_2 - \frac{3}{5}x_3 \\
 x_2 &= \frac{2}{9} + \frac{3}{9}x_1 - \frac{1}{9}x_3 \\
 x_3 &= -\frac{3}{7} + \frac{2}{7}x_1 - \frac{1}{7}x_2
 \end{aligned}$$

Choose the initial guess $x_1 = 0, x_2 = 0, x_3 = 0$

The first approximation is

$$\begin{aligned}
 x_1^{(1)} &= \frac{-1}{5} + \frac{2}{5}(0) - \frac{3}{5}(0) = -0.200 \\
 x_2^{(1)} &= \frac{2}{9} + \frac{3}{9}(0) - \frac{1}{9}(0) = 0.222 \\
 x_3^{(1)} &= -\frac{3}{7} + \frac{2}{7}(0) - \frac{1}{7}(0) = -0.429
 \end{aligned}$$

n	$k = 0$	$k = 1$	$k = 2$	$k = 3$	$k = 4$
$x_1^{(k)}$	0.000	-0.200	0.146	0.192	
$x_2^{(k)}$	0.000	0.222	0.203	0.328	
$x_3^{(k)}$	0.000	-0.429	-0.517	-0.416	

17. Show convergence in Prob. 16 by verifying that $\mathbf{I} - \mathbf{A}$, where $\mathbf{A}$ is the matrix in Prob. 16 with the rows divided by the corresponding main diagonal entries, has the eigenvalues -0.519589 and $0.259795 \pm 0.246603i$.

Bài 17: Thực hiện 5 vòng lặp với vector ban đầu $\mathbf{x}_0 = [1 \ 1 \ 1]^T$

```
>> A = [4 0 5; 1 6 2; 8 2 1]          >> b = [12.5; 18.5; -11.5]
A =                                     b =
     4     0     5                    12.5000
     1     6     2                    18.5000
     8     2     1                   -11.5000

>> [xn, count] = jacobi_method(A, b)

S =                                     T =                                     x =
     4     0     0                     0     0    -5                     1
     0     6     0                    -1     0    -2                     1
     0     0     1                    -8    -2     0                     1

iteration      iteration      iteration      iteration      iteration
it =          it =          it =          it =          it =
     1          2          3          4          5
x =           x =           x =           x =          x = 1.0e+3
    1.8750      30.0000      42.7083      342.3438      * 0.4662
    2.5833       9.9375       8.6389       86.4236       0.0695
   -21.5000    -31.6667   -271.3750   -370.4444      -2.9231
```